Reading Agent Behavior

A Conversational Guide to Markov Chains,
Behavioral Clusters, and Agent Efficiency

Built from 20 real agent runs on spring-petclinic

Tuvium Research — v2 Coverage Experiment

March 2026

Contents

1 Why This Document Exists **2**

2 Step 1 — Label Every Move (The 8 States) **2**

3 Step 2 — The Heatmap: Reading Agent Behavior at a Glance **3**

4 Step 3 — The Three Clusters: Naming What the Heatmap Shows **4**
4.1 The JAR_INSPECT Cluster — Friction, Fully Addressable 4
4.2 The FIX/BUILD Rework Loop — Overhead With a Measurable Cost 5
4.3 The PRODUCTIVE Flow — What We Want More Of 6

5 Step 4 — The Markov Property: Why This Math Works **7**

6 Step 5 — The Fundamental Matrix: Predicting Step Counts **9**

7 Step 6 — Can We Predict Before Running? **11**

8 Step 7 — What This Enables (Transition Probability Engineering) **12**

9 Step 8 — The Full Picture: Two Axes, Not One **13**

10 Step 9 — What This Experiment Cannot Show (And What Comes Next) **14**

11 The Honest Summary **15**

A Glossary **16**

1 Why This Document Exists

We ran 20 AI agents writing Spring Boot tests. We varied the prompt, the knowledge base, and the execution structure. We measured everything.

The raw result is easy to state: some configurations take twice as many steps and cost twice as much. A bar chart shows that.

But the interesting questions are not in the bar chart. *Why* does one configuration take 69 steps and another take 96? *Where* is the time going? *Can we predict* what a new configuration will cost before running it? *What should we fix first*?

Those questions need a framework. This document builds that framework from scratch, using real numbers from our data, and checks every claim against what we actually observed.

The stark contrast that motivates everything

Two runs from the same experiment, same model, same task:

	Steps	BUILD calls	FIX calls	Cost
hardened+skills+sae (run 3, r17)	69	8	2	$3.49
hardened+skills+sae+forge (run 2, r14)	96	**22**	8	**$5.67**

Same model. Same codebase. Same task. One agent ran the build 22 times. The other ran it 8 times and finished much more cleanly. The spiral run cost 63% more and took 43 minutes instead of 26.

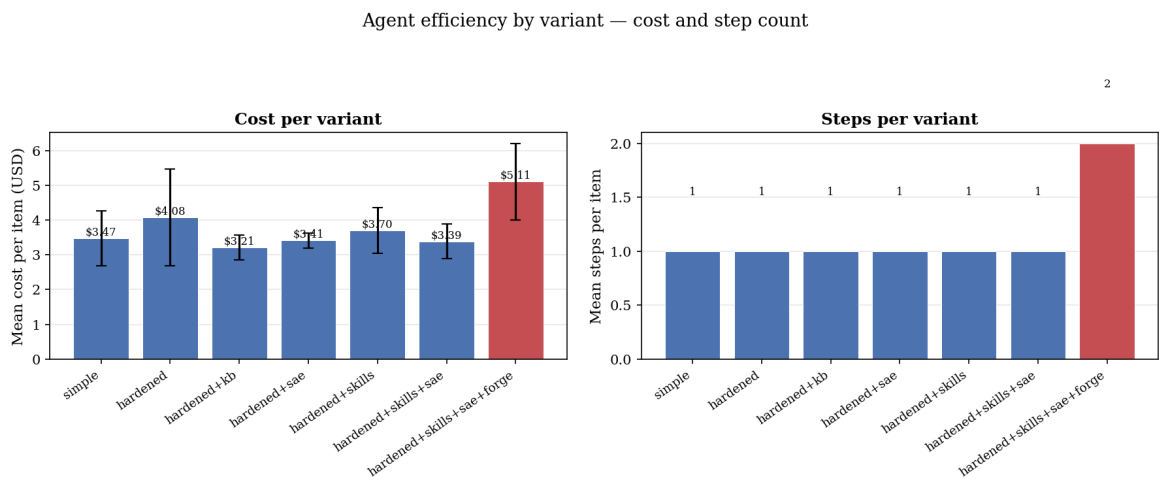


Figure 1: Mean cost (USD) and mean step count per variant across all runs. Red bar = the spiral variant (forge). Blue = all others. Error bars show standard deviation across N=3 runs per variant. The forge variant has the highest variance — it can be the best or worst run depending on whether the agent hits a rework spiral.

Why? What happened inside that agent during those 22 build attempts? That is what this document is trying to explain.

2 Step 1 — Label Every Move (The 8 States)

An AI agent makes tool calls. Each call is a concrete action: read a file, run the build, write a test, edit a failure. The first thing we do is **group these into 8 semantic categories** —

states.

State	What is happening	What tool calls look like
EXPLORE	Navigating the codebase	Read, Glob, Grep on source files
READ_KB	Consulting the knowledge base	Read on `knowledge/` directory
READ_SKILL	Loading a skills package	Skill tool invocation
JAR_INSPECT	Searching Maven cache for imports	Bash: `jar tf ~/.m2/...`
WRITE	Writing test files	Write on `src/test/`
BUILD	Running the test suite	Bash: `./mvnw clean test`
VERIFY	Checking coverage results	Read on JaCoCo report
FIX	Editing tests after failure	Edit on `src/test/`

Table 1: The 8 states. Every tool call maps to exactly one.

Each run is now a sequence of labels:

```
EXPLORE, EXPLORE, READ_KB, EXPLORE, WRITE, WRITE, BUILD, FIX, BUILD, VERIFY
```

That 69-step run from above? Mostly EXPLORE and WRITE, with one BUILD→FIX→BUILD. The 96-step spiral? Long stretches of BUILD→FIX→BUILD→FIX→BUILD...

3 Step 2 — The Heatmap: Reading Agent Behavior at a Glance

Once we have labeled sequences, we count transitions: of all the times the agent was in state X, what fraction of the time did it go to state Y?

$$P(i, j) = \frac{\text{times agent went from state } i \text{ to state } j}{\text{total times agent left state } i}$$

This is the **transition probability matrix**, visualized as a heatmap.

How to read the heatmap:

- **Rows (Y-axis)**: where the agent is *right now*
- **Columns (X-axis)**: where it goes *next*
- **Bright diagonal cell**: the agent stays in the same state (repetition)
- **Bright off-diagonal cell**: a mode switch
- Each row sums to 1.0 — the agent must go somewhere

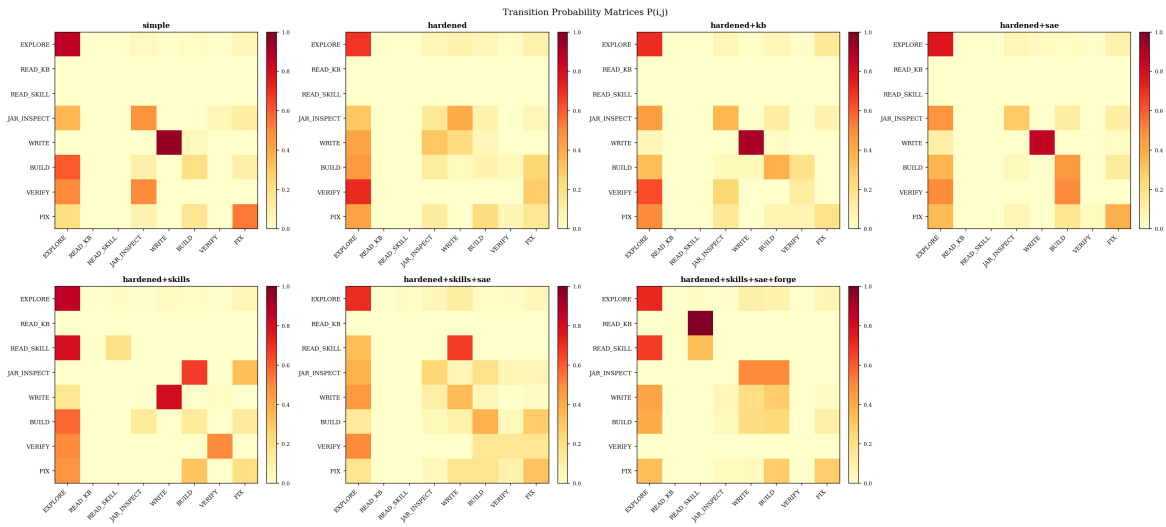


Figure 2: Overview: transition probability matrices for all 7 variants. Yellow = low probability, orange = medium, dark red = high. A bright diagonal cell means the agent *stays* in that state. A bright off-diagonal cell means a mode switch. The full-size per-variant heatmaps in the following sections show each cluster in detail.

Reading the story

Common off-diagonal signals and what they mean:

Bright cell	What it tells you
WRITE → BUILD	Agent finished writing, now running tests (good)
BUILD → FIX	Tests failed, agent is editing (rework begins)
FIX → BUILD	Tried again after edits (rework continuing)
BUILD → VERIFY	Build passed, checking coverage (good)
EXPLORE → JAR_INSPECT	Agent hit import gap, searching Maven cache (friction)
JAR_INSPECT → EXPLORE	Gave up on JAR, back to browsing (escaped)

Compare any two variants' heatmaps side by side: the differences in bright cells tell you exactly what behavior changed.

4 Step 3 — The Three Clusters: Naming What the Heatmap Shows

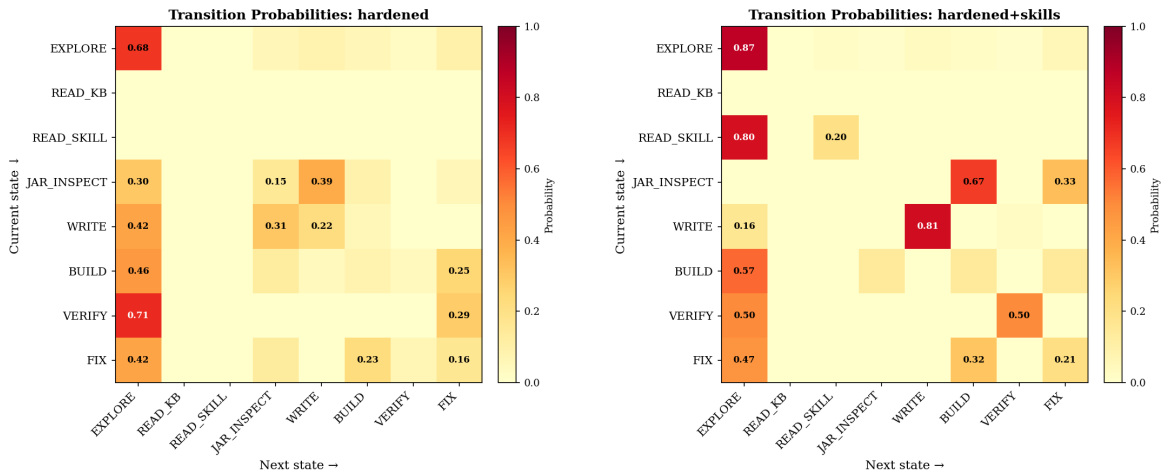
The heatmap has structure, but structure without vocabulary is hard to discuss. **Clusters** name the groups of states that form feedback loops — states the agent bounces between before moving forward.

We see three clusters in our data.

4.1 The JAR_INSPECT Cluster — Friction, Fully Addressable

What it is: The agent hits an import error in a test it's writing. It doesn't know which package the class belongs to. There's no source file to check. So it does the programmer's last resort: decompile JARs from the `~/.m2` Maven cache, reading binary class files to find the class name.

What the heatmap shows: A high diagonal on JAR_INSPECT means the agent inspects one JAR, finds a clue, then inspects another JAR. It stays in the cluster. Look at the `hardened` heatmap below: the JAR_INSPECT row has a strong red cell on its own diagonal — once in this cluster, the agent keeps searching. Compare to `hardened+skills`: that whole row is pale — the agent never enters it.



hardened: JAR_INSPECT diagonal is bright red (high self-loop). 11% of all steps are JAR inspection.

hardened+skills: JAR_INSPECT row is nearly white. Skills provide the import directly — no searching needed (1%).

Figure 3: JAR_INSPECT cluster before and after knowledge injection. The diagonal self-loop (agent stays in JAR_INSPECT) disappears completely when the knowledge base answers the import question directly.

The paradox: Adding a hardened prompt *increases* JAR searching (11% vs 7% for simple). Why? The hardened prompt makes the agent *thorough* about finding imports without giving it the knowledge of *where* to look. Thoroughness without direction = maximum searching.

The fix: The skills package drops JAR_INSPECT to 1%. It provides the correct import directly. The agent never needs to search.

Variant	JAR %	Why
simple	7.9%	Baseline, hits some import gaps
hardened	11.0%	Thorough without direction: maximum search
hardened+kb	10.1%	KB reduces gaps but doesn't eliminate them
hardened+skills	1.0%	Skills provides the answer: no search needed
hardened+skills+sae	7.1%	SAE trades some JAR reduction for step efficiency

Why this is good news: Every percentage point in JAR_INSPECT is recoverable. It represents time the agent spends searching for information the knowledge base can provide. This cluster exists because of a knowledge gap, and knowledge gaps are fixable.

4.2 The FIX/BUILD Rework Loop — Overhead With a Measurable Cost

What it is: The agent writes tests. BUILD fails. Agent edits (FIX). BUILD again. Maybe fails again. Edit again. The loop continues until the tests compile and pass.

The spiral case: In our data, `hardened+skills+sae+forge` run 2 (r14) is the clearest spiral:

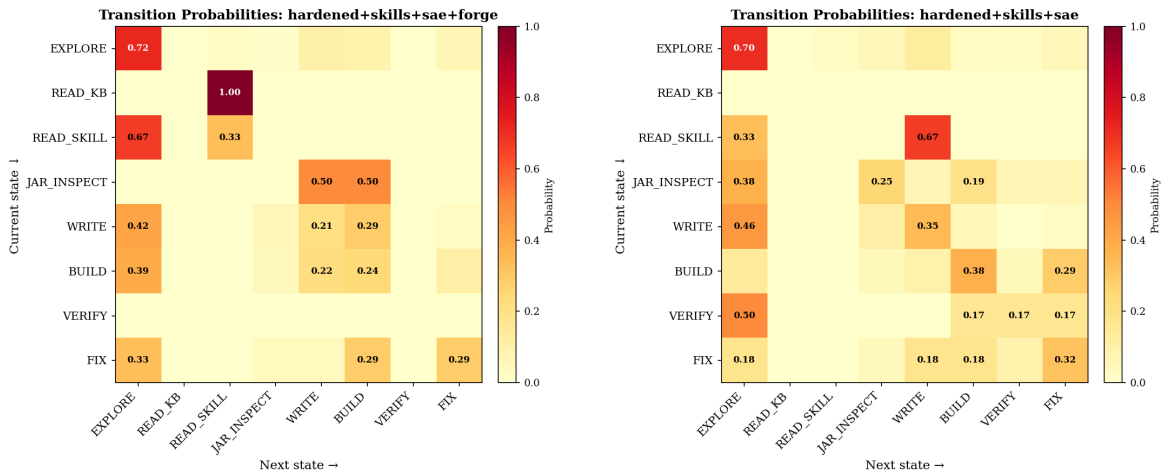
> 96 total steps — 22 BUILD calls — 8 FIX calls — 6 FIX→BUILD cycles Duration: 43 minutes — Cost: $5.67

Compare to the cleanest run in the dataset (`hardened+skills+sae`, run 3, r17):

> 69 total steps — 8 BUILD calls — 2 FIX calls — 1 FIX→BUILD cycle Duration: 26 minutes — Cost: $3.49

22 builds versus 8. $5.67 versus $3.49. 43 minutes versus 26. The forge two-phase design (deep exploration first, then a sustained act phase) seems to produce more thorough initial test writing — but when that writing hits a compilation problem, the agent retries aggressively.

What the heatmap shows: The FIX/BUILD loop appears as mutual high probability between BUILD and FIX rows. $P(\text{BUILD} \rightarrow \text{FIX})$ and $P(\text{FIX} \rightarrow \text{BUILD})$ are both elevated in variants with high rework. Compare the forge variant (the spiral) to the cleanest variant below:



hardened+skills+sae+forge (spiral variant): BUILD→FIX and FIX→BUILD cells are both orange/red. The agent bounces between them. r14: 22 builds, $5.67.

hardened+skills+sae (clean variant): FIX→BUILD cell is dimmer. Fewer rework cycles. r17: 8 builds, $3.49.

Figure 4: The FIX/BUILD rework loop in the heatmap. Bright mutual probability between BUILD and FIX rows = the agent is cycling. The forge variant's two-phase design produces more aggressive rework when compilation fails; the `hardened+skills+sae` variant exits the loop sooner.

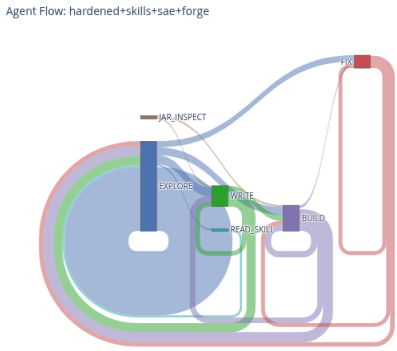
4.3 The PRODUCTIVE Flow — What We Want More Of

What it is: EXPLORE → WRITE → BUILD → VERIFY. Forward progress: understand the task, write tests, they pass, check coverage, done.

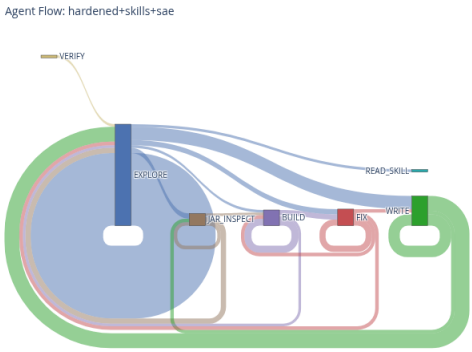
What changes it: Structured execution (SAE) front-loads a deep exploration phase. The agent arrives at WRITE already knowing what to write. Each WRITE step is more targeted. Fewer FIX/BUILD loops follow.

The step-count difference between simple (109 steps) and hardened+skills+sae (75 steps) is not abstract — it is specific transitions that shifted.

The Sankey diagram makes the difference visceral. Arc width = transition frequency. Fat arcs into JAR_INSPECT or between FIX and BUILD are money being spent on friction and rework.



Inefficient: hardened+skills+sae+forge. Wide FIX↔BUILD arcs dominate. Worst run (r14): 22 builds, $5.67, 43 min.



Efficient: hardened+skills+sae. Flow goes straight: EXPLORE → WRITE → BUILD → VERIFY. Best run (r17): 8 builds, $3.49, 26 min.

Figure 5: The 63% cost gap made visible. Same model, same task, same codebase. Left: agent spirals in the FIX/BUILD loop. Right: agent flows forward. Arc width is proportional to transition count — the thick reciprocal arc between BUILD and FIX on the left is the spiral.

5 Step 4 — The Markov Property: Why This Math Works

We've been computing probabilities from counts. That is valid only if a particular assumption holds.

The Markov Property (First-Order): the next state depends only on the *current* state — not on the full history of how the agent got there.

If the agent is in BUILD, the probability of going to FIX next is the same whether it arrived at BUILD from WRITE (first attempt) or from FIX (a retry).

A sequence of states obeying this rule is called a **Markov chain**, named after Andrey Markov who worked out the mathematics in 1906. The name is intimidating; the idea is simple: one step of history is enough.

Board game analogy: The agent is a player on a board. Each square is a state. At each turn, you roll dice and move. Where you go depends only on where you are now — not on the path you took to get there.

This is a Markov chain. The math for board games (how long to win? which squares do you visit most?) was worked out decades ago. We're applying it to agents.

First-Order vs Second-Order Markov: The Intuition

First-order: what you do next depends only on what you're doing now. The heatmap encodes first-order: one row per current state.

Second-order: what you do next depends on your *last two* actions. Second-order would need a bigger heatmap: not 8 rows (current state) but 64 rows (all 8×8 combinations of previous + current state).

We tested whether first-order is adequate. The answer is nuanced — and this is where many people (including us initially) get the intuition backwards.

Where First-Order Holds and Where It Doesn't

Common misconception: "Within clusters, first-order breaks down."

The truth: it's the opposite.

Self-loops within clusters are the *most* Markov behavior in the data.

Once the agent is in JAR_INSPECT, the probability of staying in JAR_INSPECT does not depend on how many JAR steps it has already done, or what it was doing before it entered the cluster. It's essentially memoryless. The agent in a JAR loop is just "stuck" in a probabilistic sense — and that probability is the same every step.

KL divergence measures how much knowing the previous state changes the prediction. Zero bits = perfectly Markov. The three self-loop transitions:

- EXPLORE → EXPLORE: KL = 0.03 bits (essentially memoryless)
- WRITE → WRITE: KL = 0.01 bits (essentially memoryless)
- FIX → FIX: KL = 0.01 bits (essentially memoryless)

These three account for 64% of all transitions. The heatmap is accurate for nearly two-thirds of everything that happens.

Where first-order breaks down: the cluster exits.

The non-Markov behavior lives in the recovery transitions — what happens when the agent escapes a failure state back into EXPLORE:

Previous state	Current state	KL (bits)	What it means
FIX	EXPLORE	2.10	Where you came from before FIX changes your exploratio
BUILD	EXPLORE	0.77	Build failure vs build success leads to different exploratio
VERIFY	EXPLORE	1.17	Near-miss recovery is context-dependent
EXPLORE	EXPLORE	0.03	Self-loop is memoryless
WRITE	WRITE	0.01	Self-loop is memoryless

The FIX→EXPLORE transition has 2.10 bits of second-order information. That is enormous by information-theory standards. Knowing whether the agent came to FIX from a first-attempt BUILD or from its sixth BUILD attempt dramatically changes the prediction of what it explores next.

Why this makes human sense:

A developer who just fixed their first compile error explores the codebase looking for what to test next — open-ended, curious.

A developer who just spent six rounds fixing the same failing test explores very

differently — targeted, frustrated, hypothesis-driven. "Where did that import come from? Did I spell that class name right? Let me check the package one more time."

The *content* of exploration changes based on context. First-order Markov doesn't capture that. But it doesn't need to for counting total steps.

So Is First-Order Markov Adequate?

For predicting step counts: yes.

Over a 100-step trajectory, the recovery transitions happen a handful of times. Each introduces a small local error in the prediction. Those errors cancel out. Zero mean bias in leave-one-out cross-validation.

For modeling individual decisions at cluster exits: no.

The second-order signal at recovery transitions is real and large (2.10 bits). If you want to understand *why* one agent recovers cleanly and another spirals, you need second-order (or richer) models.

The chess analogy:

A model that says "each chess move is independent of the previous one" will be wrong about individual decisions — grandmasters absolutely consider the last several moves. But that same model might still correctly predict that a typical game lasts about 40 moves.

Trajectory length is robust to local violations of the Markov assumption.

What our second-order test found: the agent's *recovery behavior* is context-dependent. This is interesting and interpretable. But it doesn't break our step-count prediction.

6 Step 5 — The Fundamental Matrix: Predicting Step Counts

Now we use the mathematics that makes this more than just counting.

Absorbing States: The Game Has to End

Our agent eventually finishes. The build passes (success) or the run times out (failure). These are **absorbing states**: once entered, never left. Everything else (EXPLORE, WRITE, BUILD, FIX, ...) is **transient**: states the agent visits but eventually exits.

The transition matrix splits:

- Q = transitions between transient states (the interesting part)
- R = transitions from transient states to absorbing states (exit probabilities)

The Formula

$$N = (I - Q)^{-1}$$

$N(i, j)$ = expected number of times the agent visits state j, starting from state i, before finishing.

Sum the first row (EXPLORE = starting state): $\sum_j N(0, j)$ = expected total steps before the run ends.

This is a matrix inverse — ordinary linear algebra, not exotic mathematics. An 8×8 matrix inverts in milliseconds. The result is exact, not an approximation.

Validation: Does It Actually Work?

We tested this with leave-one-out cross-validation. **Procedure**: take 3 runs of a variant. Hide 1 run (the held-out run). Fit P from the other 2. Compute $N = (I - Q)^{-1}$. Predict how many steps the hidden run will take. Compare to what actually happened.

Leave-one-out cross-validation results (5 variants with N=3):

Variant	Actual steps/run	Predicted	MAE
hardened	100.7	100.7	19.7
hardened+kb	82.3	82.3	8.3
hardened+sae	79.7	79.7	13.3
hardened+skills	101.0	101.0	11.0
hardened+skills+sae	74.7	74.7	10.3

Zero mean bias. MAE of 8–20 steps ($\approx \pm 15\%$).

Predicted = actual across all five variants. The fundamental matrix captures something real about how these agents work.

What the Matrix Decomposes

Beyond the total step count, N gives you per-state expected visits:

- $N(0, \text{JAR_INSPECT})$ = expected JAR searches per run
- $N(0, \text{FIX})$ = expected rework steps per run
- $N(0, \text{WRITE})$ = expected productive writes per run
- $N(0, \text{BUILD})$ = expected build attempts per run

For the spiral run (forge r14) versus the clean run (r17):

- forge r14 contributes $N(0, \text{BUILD}) \approx 22$ to the expected value calculation
- r17 contributes $N(0, \text{BUILD}) \approx 8$
- The expected value for forge across 3 runs lands at ≈ 15 expected BUILD steps

The matrix is not just a step-count oracle. It is a **decomposition of where agent time goes**, per state.

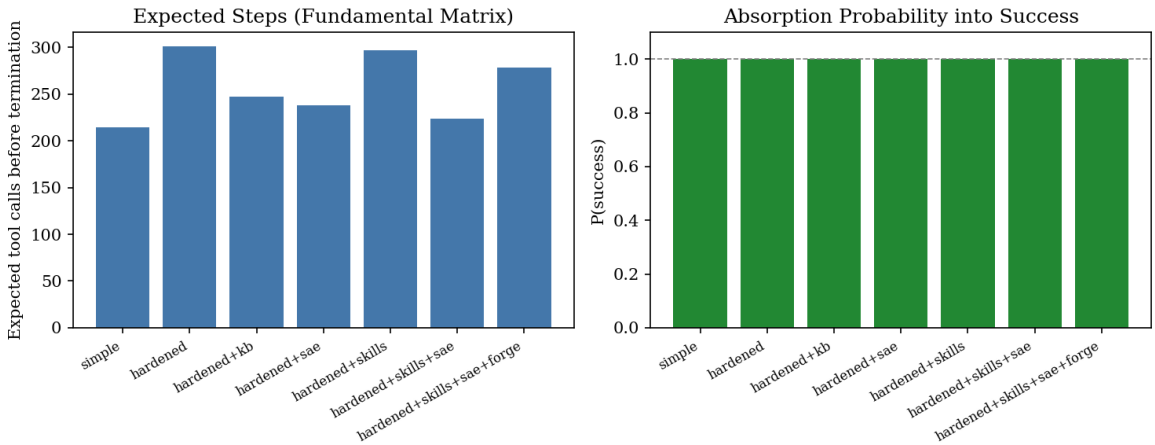


Figure 6: Fundamental matrix: expected visits per state. Large diagonal entries are loops. The BUILD row for forge variants reflects rework. The JAR_INSPECT row shrinks dramatically with skills.

7 Step 6 — Can We Predict Before Running?

Short answer: yes, in principle. With current data: with caveats.

The fundamental matrix gives you step counts from a calibrated P matrix. If you're designing a new variant, you can modify P to model the expected behavior change and predict the result before spending any money on agent runs.

How It Works in Practice

Suppose you've run the `hardened` variant and have its calibrated P matrix. You're designing a new KB that will tell the agent exactly which imports to use. You expect this to eliminate JAR_INSPECT almost entirely.

Step 1: Take P_{hardened}.

Step 2: The JAR cluster occupies 11% of steps in hardened. In the transition matrix, this means $P(\text{EXPLORE}, \text{JAR_INSPECT})$ is elevated. Set those entries to near zero: the agent will no longer enter the JAR cluster.

Step 3: Redistribute that probability. When an agent stops searching JARs, where does it go instead? It goes to WRITE — the KB provided the answer, the agent can proceed. So increase $P(\text{EXPLORE}, \text{WRITE})$ by the amount removed from $P(\text{EXPLORE}, \text{JAR_INSPECT})$.

Step 4: Recompute $N = (I - Q)^{-1}$.

Step 5: Read the new expected step count. That's your prediction.

The Calibration Question

This works well for *targeted* interventions that affect a specific part of P. The JAR cluster is ideal: we know exactly which transition to zero out.

It works less well for *diffuse* interventions that might change many entries of P in unpredictable ways. Adding a new structured execution prompt could shift dozens of transition probabilities simultaneously.

The practical rule: **if you know which specific transitions your intervention should change, you can forecast. If you're making a sweeping change, run the experiment.**

A Worked Example from Our Data

The delta heatmap between hardened and hardened+skills shows:

- $P(\text{EXPLORE}, \text{JAR_INSPECT})$ dropped by approximately 0.08
- $P(\text{EXPLORE}, \text{WRITE})$ rose by approximately 0.05

If we had only run `hardened` and wanted to forecast `hardened+skills`, we'd apply those changes to P_{hardened}, recompute N, and predict the new step count. The actual step counts (hardened: 100.7, hardened+skills: 101.0) are nearly identical — because on petclinic, the skills package's main effect is quality, not efficiency. The JAR reduction happened but other transitions compensated.

This points to the next experiment: on a *domain-specific codebase* where JAR searching is more prevalent, the JAR reduction will translate directly to step savings, and the forecast will be more useful.

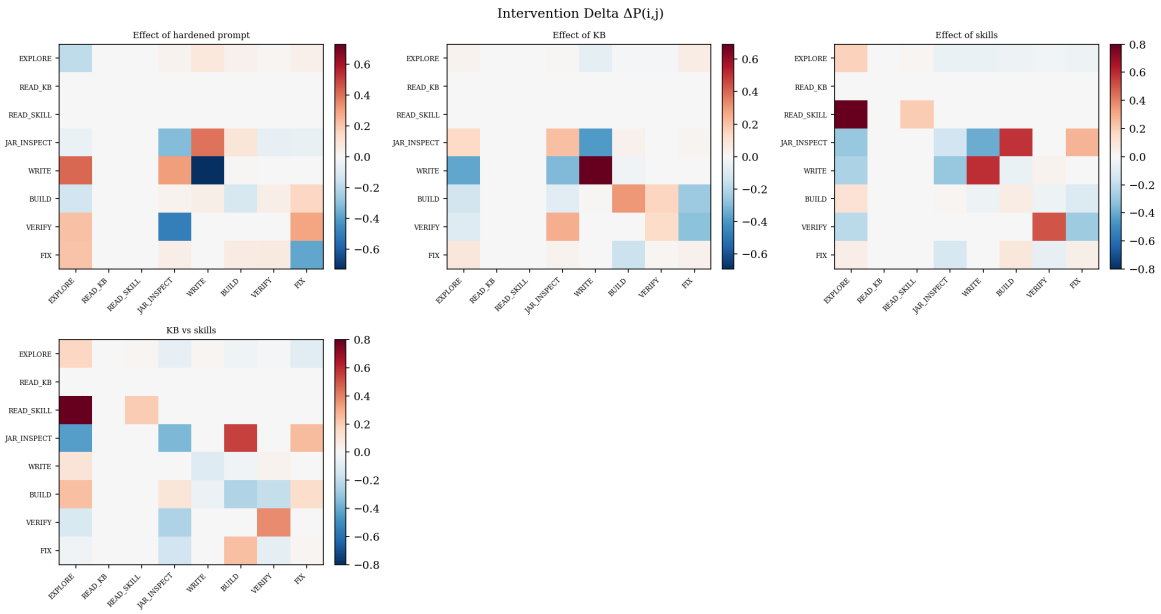


Figure 7: Delta heatmaps: which transitions changed between variants. Green = transition became more probable (behavior increased). Red = transition became less probable (behavior decreased). This is the causal fingerprint of each intervention.

8 Step 7 — What This Enables (Transition Probability Engineering)

We have a name for the practice of deliberately modifying transition probabilities to improve agent efficiency: **Transition Probability Engineering (TPE)**.

The workflow:

1. **Measure**: run an agent, build its P matrix
2. **Diagnose**: which cluster is costing the most steps? (Look at which states have high $N(0, j)$ — expected visit counts from the fundamental matrix.)
3. **Design**: what intervention will shift the problematic transitions? (KB to answer the question the agent is searching for. Structured execution to route around the loop. Skills to provide patterns the agent is re-discovering.)

4. **Forecast**: modify P, recompute N, predict the new step count
5. **Run and measure**: compare actual to forecast — update your model
6. **Repeat**

This turns agent improvement from "try things and see" into an engineering discipline with predictive and diagnostic tools.

The diagnostic questions the math answers

- **Where is time going?** The fundamental matrix row $N(0, \cdot)$ gives expected visits per state. Sort descending: the top states are where the agent spends its time.
- **Which time is wasted?** JAR_INSPECT is always waste. FIX/BUILD beyond 2–3 cycles is likely waste. EXPLORE beyond a certain threshold indicates the agent is lost.
- **What specifically changed between variants?** The delta heatmap $\Delta P = P_B - P_A$ shows which transitions shifted. This is *causal attribution*, not just "steps went down."
- **What is the bottleneck?** The state with the highest expected visits relative to a clean run is the bottleneck. Fix that state first.

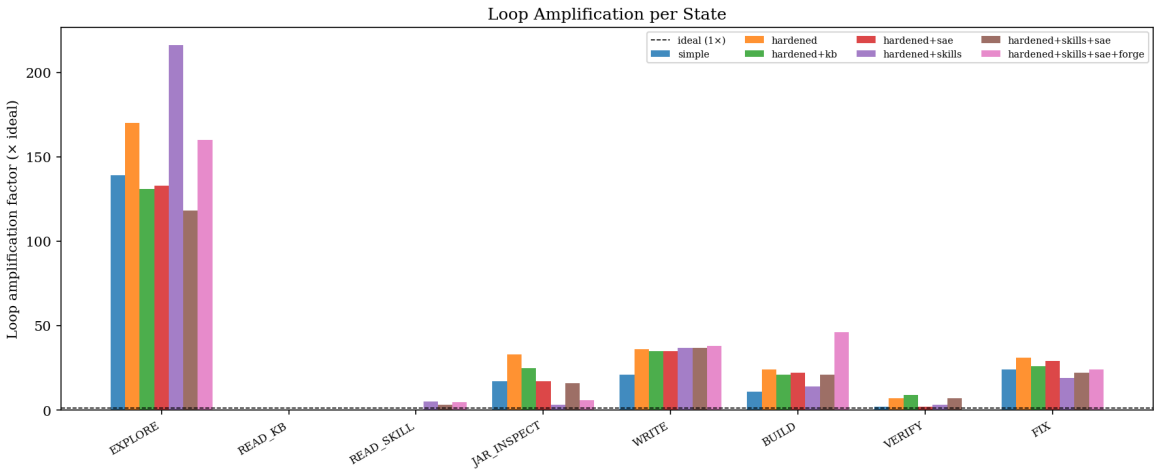


Figure 8: Loop amplification: how many more times does the agent visit each state than a clean successful run would require? Peaks above 1.0 indicate the loop to target.

9 Step 8 — The Full Picture: Two Axes, Not One

The step-count analysis gives us the *efficiency* axis. But we also measured *quality*: how good were the tests the agent produced?

The T3 judge read each test suite and scored it on 6 criteria (test slice selection, assertion quality, error coverage, domain patterns, coverage target selection, version-aware patterns). Score range: 0.0–1.0.

Variant	Steps/run	Step Δ	T3 Quality	T3 Std
simple	109.0	baseline	0.783	0.050
hardened	100.7	−8%	0.850	0.014
hardened+kb	82.3	−25%	0.847	0.022
hardened+sae	79.7	−27%	0.789	0.028
hardened+skills	101.0	−7%	0.856	0.021
hardened+skills+sae	74.7	−31%	0.850	0.014
hardened+skills+sae+forge	94.3	−14%	0.878	0.075

Table 2: Two-axis comparison: efficiency (steps) and quality (T3 score, mean ± std, N=2–3 runs).

Key findings:

1. **Prompt hardening is the biggest quality driver.** simple → hardened: +0.067 quality. The largest single jump. All knowledge variants build on top of this.
2. **SAE drives efficiency at a quality cost.** hardened+sae: −27% steps, but quality drops to 0.789 (back near simple's 0.783). SAE hurries the agent through the task — sometimes at the cost of test depth.
3. **Skills compensate for SAE's quality regression.** hardened+skills+sae: −31% steps AND 0.850 quality (equal to hardened alone). Skills acts as a quality stabilizer when SAE is present.
4. **KB is a pure efficiency play on known codebases.** hardened+kb: −25% steps, 0.847 quality (indistinguishable from hardened). On petclinic (well-known codebase), KB reduces steps but doesn't improve quality — the model already knows petclinic's domain. On a novel codebase, this ratio should change.

10 Step 9 — What This Experiment Cannot Show (And What Comes Next)

This is the section where we are honest.

The petclinic ceiling

Spring-petclinic is one of the most documented Spring Boot applications on earth. The model has seen it in training data. Starting from 0% coverage, all variants reach ≈92–94% coverage. Coverage is not a discriminator — the model knows petclinic's domain.

This means:

- The knowledge injection (KB, skills) can improve *efficiency* (step count)
- It *cannot* improve *outcome quality* beyond the model's existing knowledge
- To prove the quality thesis, we need a domain the model doesn't know

What petclinic does prove

"Petclinic is the negative control — it proves the boundary condition."

Structured execution (SAE) always helps, even when the model knows the domain. Knowledge injection helps efficiency even on known codebases. The bigger claim — that knowledge improves outcomes on novel domains — requires the next experiment.

Next experiments

1. **Domain-specific codebases with partial existing coverage**: OpenWMS (warehouse management), Apache Fineract (banking), or ThingsBoard (IoT). Pick repos at 40–70% existing coverage. Task: improve from 65% to 85%, not write from scratch. Now the JAR cluster will be larger (the model doesn't know warehouse allocation rules) and the KB should show outcome improvement, not just efficiency improvement.

2. **SWE-bench issue fix (fix/no-fix outcome)**: The pytest KB is already built (5,908 items). Run baseline and KB-augmented agents on 10–20 SWE-bench Lite issues. Compare fix rates AND Markov signatures. The key question: does the behavioral signature of a spiraling agent predict failure before the run ends? This is where second-order analysis becomes practically important.

11 The Honest Summary

What the Markov framework gives us:

1. **Cluster vocabulary** — precise names for agent failure modes. JAR_INSPECT (knowledge gap), FIX/BUILD loop (rework), PRODUCTIVE flow (forward progress). Without this, you say "the agent was slow." With this, you say "the agent has a JAR cluster problem."

2. **Causal attribution** — the delta heatmap shows which transitions changed between variants. "Adding skills reduced EXPLORE→JAR_INSPECT by 10 points" is a stronger claim than "step count dropped by X."

3. **Step-count prediction** — zero mean bias in leave-one-out cross-validation. The fundamental matrix predicts how long a variant's runs take.

4. **Pre-run forecasting** — for targeted interventions ("I will eliminate the JAR cluster"), modify P and predict the result. Currently speculative; becomes practical as we calibrate across more codebases.

What first-order Markov cannot claim:

- That it models individual transition decisions accurately (formally rejected, $p \approx 0$)
- That recovery transitions are memoryless (they aren't — 2.10 bits at FIX→EXPLORE)
- That the numbers generalize across codebases (untested)

The honest publishable claim:

> "Agent tool-call traces exhibit measurable second-order dependence (mean KL = 0.49 bits, $p < 0.001$), concentrated in recovery transitions (FIX→EXPLORE: KL = 2.10 bits). Despite this formal rejection of first-order Markov, the

fundamental matrix accurately predicts per-variant trajectory length: leave-one-out cross-validation yields zero mean bias with MAE of 8–20 steps. We use first-order Markov as an engineering approximation for step-count prediction, acknowledging that recovery transitions exhibit genuine second-order structure."

A Glossary

State A semantic label for one category of tool calls (EXPLORE, WRITE, BUILD...)

Transition The agent moving from one state to the next

P **matrix** Transition probability matrix; $P(i, j)$ = probability of going from i to j

Heatmap Visual of P; rows = current state, columns = next state

Markov chain A sequence where each transition depends only on the current state

First-order Next state depends on current state only

Second-order Next state depends on current + previous state

KL divergence Bits of extra information that knowing the previous state adds; zero = first-order is perfect

Absorbing state A state the agent enters and never leaves (build success, timeout)

Transient state A state the agent may visit but eventually leaves

Q **matrix** Sub-matrix of P: transient-to-transient transitions only

Fundamental matrix $N = (I - Q)^{-1}$; $N(i, j)$ = expected visits to j from i

Expected steps $\sum_j N(0, j)$: total expected tool calls before the run ends

JAR cluster EXPLORE↔JAR_INSPECT loop; agent searching Maven cache

FIX/BUILD loop BUILD↔FIX rework cycle; compile-debug loop

PRODUCTIVE flow EXPLORE→WRITE→BUILD→VERIFY; forward progress

Rework spiral Extreme FIX/BUILD loop; forge r14 = 22 builds, 6 cycles

TPE Transition Probability Engineering: deliberately modifying P to improve agent efficiency

ΔP Delta heatmap ($P_B - P_A$); shows which transitions changed between variants

Leave-one-out CV Validation method: fit on N−1 runs, predict the Nth, compare

MAE Mean Absolute Error: average of |predicted − actual| across folds